

2D Bin-Packing Problem Report

AIRPORT AND CARGO OPERATIONS

AE4446

Authors:

Job De Vries (5175879)
Júlia Capella Campdepadrós (6281540)
Marianna Piperigou Grammatika (5236398)

February 17, 2025



Contents

1	Introduction	1
2	Mathematical Model	2
2.1	Problem Definition	2
2.2	Sets, Parameters, and Decision Variables	2
2.3	Mathematical Formulation	3
3	Packing Results	6
4	Algorithmic Results	7
5	Managerial Considerations	8
6	Extension	8
7	Personal Reflections	11
7.1	Job	11
7.2	Marianna	11
7.3	Júlia	12
	References	13

1 Introduction

In this report, a 2D Bin Packing Problem (2DBPP) will be presented for an air cargo application. The aim of this assignment is to optimize the packing strategy for a set of items to be transported in the belly space of a passenger aircraft, into available ULDs (Unit Load Devices) while minimizing the cost of the ULDs used. The problem is taken from literature [1], simplified from a 3DBPP, and presented in a vertical 2D plane, where the depth dimension of both the items and ULDs is neglected, and only the horizontal (x) and vertical (z) directions are considered. The challenge is to determine the most efficient packing arrangement that fits all items into the available ULDs, subject to constraints in type and rotation allowances. The model will be created in Python and solved using Gurobi branch and bound method with a 2 hour time limit.

The report is structured as follows. First, the 2DBPP problem definition and mathematical model formulation will be presented in [section 2](#). Then, the results visualization of the optimization done using Gurobi will be shown in [section 3](#). The algorithmic results with the final solution will be presented in [section 4](#). This result will be discussed and its real world managerial application will be analyzed in [section 5](#). The problem will be extended to account for a more accurate real-life scenario and presented once more in [section 6](#). Finally, the teams personal reflections on the assignment will be given in [section 7](#).

2 Mathematical Model

2.1 Problem Definition

The 2D Bin Packing Problem (2DBPP) is a fundamental combinatorial optimization problem that has widespread applications in logistics, cargo loading, manufacturing, and supply chain management. The objective is to pack a given set of rectangular items into the smallest number of bins or minimize the total bin cost while satisfying a series of constraints such as bin dimensions, item orientations, and separation conditions with a Mixed Integer Linear Programming (MILP) approach.

In air transport and cargo operations, optimizing space utilization is crucial for reducing costs and ensuring that items are arranged in a stable and accessible manner. The problem becomes even more complex when additional real-world constraints, such as perishable and radioactive item separation and fragility considerations are introduced. As a result, developing a robust mathematical model to handle these constraints while maintaining computational efficiency is an important challenge.

To solve the 2D-BPP, the Gurobi optimizer in Python is utilized. All standard settings are used besides the time limit which has been set to 2 hours.

2.2 Sets, Parameters, and Decision Variables

Sets

- I : Set of items, indexed by i .
- J : Set of bins, indexed by j .
- $D = \{0, 1\}$: Set of coordinate axes (0 = x-axis, 1 = z-axis), indexed by d .
- $O = \{0, 1\}$: Set of edge orientations (0 = original¹, 1 = rotated), indexed by c .

Parameters

- L_j : Length of bin j .
- H_j : Height of bin j .
- $A_j = L_j \cdot H_j$: Area of bin j .
- C_j : Cost of using bin j .
- l_i : Length of item i .
- h_i : Height of item i .
- $A_i = l_i \cdot h_i$: Area of item i .
- a_j, b_j : Parameters defining a cut in bin j (if applicable).
- $M = 10000$: Large constant used for big-M constraints.
- $L = M$: Large constant used for big-M constraints (pertaining to horizontal coordinate constraints)².
- $H = M$: Large constant used for big-M constraints (pertaining to vertical coordinate constraints).
- $per_i \in \{0, 1\}$: Perishability of item i , 1 if item is perishable, else 0.
- $rad_i \in \{0, 1\}$: Radioactivity of item i , 1 if item is radioactive, else 0.
- $fra_i \in \{0, 1\}$: Fragility of item i , 1 if item is radioactive, else 0.

Decision Variables

- $p_{ij} \in \{0, 1\}$: 1 if item i is placed in bin j , 0 otherwise.
- $u_j \in \{0, 1\}$: 1 if bin j is used, 0 otherwise.

¹Long side aligned with horizontal axis

²section 4 will explain why this was chosen instead of $\max(L_j)$ or $\max(H_j)$

- x_i : x-coordinate of the bottom-left corner of item i .
- z_i : z-coordinate of the bottom-left corner of item i .
- x'_i : x-coordinate of the top-right corner of item i .
- z'_i : z-coordinate of the top-right corner of item i .
- $r_{i,d,o} \in \{0, 1\}$: 1 if item i is placed with orientation o along axis d , 0 otherwise.
- $x_{ik}^p \in \{0, 1\}$: 1 if item i is to the right of item k .
- $z_{ik}^p \in \{0, 1\}$: 1 if item i is above item k .
- $\gamma_i \in \{0, 1\}$: 1 if item i is supported by a bin cut.
- $g_i \in \{0, 1\}$: 1 if item i is placed on the ground of a bin.
- $h_{ik} \in \{0, 1\}$: 1 if item k has a suitable height to support item i ($z_i = z'_k$).
- $o_{ik} \in \{0, 1\}$: 1 if item k has a non-empty intersection on the x-axis with item i .
- $s_{ik} \in \{0, 1\}$: 1 if item k supports item i and they are in the same bin.
- $\eta_{1,ik} \in \{0, 1\}$: 1 if vertex 1 of item k is to the left of item i ($x_k \leq x_i$).
- $\eta_{2,ik} \in \{0, 1\}$: 1 if vertex 2 of item j is to the right of item i ($x'_k \leq x'_i$).
- $\beta_{1,ij} \in \{0, 1\}$: 1 if vertex 1 of item i is supported by item j .
- $\beta_{2,ij} \in \{0, 1\}$: 1 if vertex 2 of item i is supported by item j .
- $v_{ik} \in \mathbb{R}^+$: Absolute value of $z'_k - z_i$.
- $m_{ik} \in \{0, 1\}$: 1 if item k overlaps or exceeds item i in height, i.e., $z'_k > z_i$.
- $pb_j \in \{0, 1\}$: 1 if bin j contains perishable items.
- $rb_j \in \{0, 1\}$: 1 if bin j contains radioactive items.

2.3 Mathematical Formulation

Objective:

$$\min \sum_{j \in J} C_j u_j. \quad (1)$$

That is, we minimize the total cost of bins used.

Subject to:

(C1) Assignment Constraint: Each item i is assigned to exactly one bin:

$$\sum_{j \in J} p_{ij} = 1, \quad \forall i \in I. \quad (2)$$

(C2) Flagging Constraint: For each item i and bin j , ensure that the flag variable $u_j[j]$ is set to 1 if bin j is used:

$$u_j \geq p_{ij}, \quad \forall i \in I, \quad j \in J. \quad (3)$$

(C3) Area Constraint: The total area of items assigned to bin j must not exceed the area of bin j , and it only applies when bin j is used:

$$\sum_{i \in I} a_i \cdot p_{ij} \leq A_j \cdot u_j, \quad j \in J. \quad (4)$$

(C4) Fit-in-Bin Constraints: Ensure that each item i fits within the dimensions of its assigned bin:

$$x'_i \leq \sum_{j \in J} L_j p_{ij}, \quad z'_i \leq \sum_{j \in J} H_j p_{ij}, \quad \forall i \in I. \quad (5)$$

(C5) Orientation Constraints: The dimensions of item i are determined by its orientation:

$$x'_i - x_i = \sum_{d \in D} r_{i,0,d} \ell_i^{(d)}, \quad z'_i - z_i = \sum_{d \in D} r_{i,1,d} \ell_i^{(d)}, \quad (6)$$

with $\ell_i^{(1)} = l_i$ and $\ell_i^{(2)} = h_i$. In addition, each side of an item may only be aligned with one axis:

$$\sum_{c \in C} r_{i,c,d} = 1, \quad \forall i \in I, \quad d \in D. \quad (7)$$

Similarly, each bin axis may only be aligned with one side of each item:

$$\sum_{d \in D} r_{i,c,d} = 1, \quad \forall i \in I, \quad c \in O. \quad (8)$$

(C6) Length-side orientation constraint: The length side of an item can only be aligned along the vertical axis if rotation is allowed:

$$r_{i,1,0} \leq l_i^+, \quad \forall i \in I. \quad (9)$$

(C7) Height-side orientation constraint: The height side of an item can only be aligned along the horizontal axis if rotation is allowed:

$$r_{i,0,1} \leq l_i^+, \quad \forall i \in I. \quad (10)$$

(C8) Non-overlap Constraints: Ensures that two items do not overlap in the same bin:

$$x_{ik}^p + x_{ki}^p + z_{ik}^p + z_{ki}^p \geq (p_{ij} + p_{kj}) - 1, \quad \forall i, k \in I, j \in J, \quad i \neq k. \quad (11)$$

(C9) Right-overlap condition: If item i is to the right of item k , then the right boundary of k must be smaller than or equal to the left boundary of i :

$$x'_k \leq x_i + (1 - x_{ik}^p)L, \quad \forall i, k, \quad i \neq k. \quad (12)$$

(C10) Not Right-overlap Constraint: For items i and k , the left boundary of item i plus a small value ϵ must be less than or equal to the right boundary of item k , unless the items are related by $x_p[i, k]$:

$$x_i + \epsilon \leq x'_k + x_{ik}^p \cdot H, \quad \forall i, k \in I. \quad (13)$$

(C11) Above-overlap condition: If item i is above item k , then the upper boundary of k must be smaller than or equal to the lower boundary of i :

$$z'_k \leq z_i + (1 - z_{ik}^p)H, \quad \forall i, k, \quad i \neq k. \quad (14)$$

(C12) Not Above-overlap Constraint: For items i and k , the lower boundary of item i plus a small value ϵ must be less than or equal to the upper boundary of item k , unless the items are related by $z_p[i, k]$:

$$z_i + \epsilon \leq z'_k + z_{ik}^p \cdot H, \quad \forall i, k \in I. \quad (15)$$

(C13) Cut Support Constraint: Ensures that items located in a bin with a cut are always above the cut.

$$z_i + \frac{b_j}{a_j} x_i - b_j \geq -M(1 - p_{ij}), \quad \forall i \in I, \quad \forall j \in J_{cut}. \quad (16)$$

(C14) Vertical Stability Constraint: An item must be supported by at least two points by either the cut, another item or the ground.

$$\gamma_i + \sum_{j \in I, j \neq i} \beta_{1,ij} + \sum_{j \in I, j \neq i} \beta_{2,ij} + 2g_i \geq 2, \quad \forall i \in I. \quad (17)$$

(C15) Ground Placement Constraint: Ensures that an item can only be placed on the ground if z_i is 0.

$$z_i \leq (1 - g_i)H, \quad \forall i \in I. \quad (18)$$

(C16) Absolute Height Difference Constraints: Defines the absolute difference in height between items.

$$z'_k - z_i \leq v_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (19)$$

$$z_i - z'_k \leq v_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (20)$$

(C17) Relative Vertical Placement Constraints: Defines whether an item i is above an item k or not.

$$v_{ik} \leq z'_k - z_i + 2H(1 - m_{ik}), \quad \forall i, k \in I, \quad i \neq k. \quad (21)$$

$$v_{ik} \leq z_i - z'_k + 2Hm_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (22)$$

(C18) Height Compatibility Constraints: Ensures items are only be supported if their heights align.

$$h_{ik} \leq v_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (23)$$

$$v_{ik} \leq h_{ik}H, \quad \forall i, k \in I, \quad i \neq k. \quad (24)$$

(C19) Horizontal Overlap Constraints: Ensures that the non-empty horizontal intersection variable o_{ik} is flagged to 0 if neither items are to the right of each other and vice versa.

$$o_{ik} \leq x_{ik}^p + x_{ki}^p, \quad \forall i, k \in I, \quad i \neq k. \quad (25)$$

$$x_{ik}^p + x_{ki}^p \leq o_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (26)$$

(C20) Support Constraints: Ensure items are supported if item i and k have a non empty intersection and suitable vertical locations.

$$(1 - s_{ik}) \leq h_{ik} + o_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (27)$$

$$h_{ik} + o_{ik} \leq 2(1 - s_{ik}), \quad \forall i, k \in I, \quad i \neq k. \quad (28)$$

(C21) Support Propagation Constraints: Ensure support indicators are correctly flagged.

$$\beta_{1,ik} \leq s_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (29)$$

$$\beta_{2,ik} \leq s_{ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (30)$$

(C22) Item Placement Constraints: Ensure item alignment for support.

$$\eta_{1,ik} \leq 1 - \beta_{1,ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (31)$$

$$\eta_{2,ik} \leq 1 - \beta_{2,ik}, \quad \forall i, k \in I, \quad i \neq k. \quad (32)$$

(C23) Fragility Constraint: Prevents stacking items on fragile ones.

$$s_{ik} \leq n_{items}(1 - \text{fra}_k), \quad \forall i, k \in I, \quad i \neq k. \quad (33)$$

(C24) Bin Perishability and Radioactivity Flagging Constraint: If a perishable item i is placed in bin j , the bin perishability variable per_j will be flagged (same for radioactivity).

$$\sum_{\{i \in I \mid per_i=1\}} p_{ij} \leq M \cdot pb_j, \quad \forall j \in J, \quad (34)$$

$$\sum_{\{i \in I \mid rad_i=1\}} p_{ij} \leq M \cdot rb_j, \quad \forall j \in J, \quad (35)$$

To ensure the bin perishability and radioactivity variables do not get activated even when there are not any perishable or radioactive items present in the bin, the reverse must also hold:

$$pb_j \leq \sum_{\{i \in I \mid per_i=1\}} p_{ij}, \quad \forall j \in J, \quad (36)$$

$$rb_j \leq \sum_{\{i \in I \mid rad_i=1\}} p_{ij}, \quad \forall j \in J, \quad (37)$$

(C25) Perishable and Radioactive Linking Constraint: Ensures that perishable and radioactive items do not coexist in the same bin j .

$$pb_j + rb_j \leq 1, \quad \forall j \in J. \quad (38)$$

3 Packing Results

The best solution obtained after two hours for the described problem described in the previous sections is showed in Figure 1. The color of each item is determined by its rotability, and the borders are red if the item is fragile. Additionally, items with grid lines are either radioactive or perishable, depending on the color. Finally, the four labeled numbers indicate whether the item is rotatable, fragile, perishable, and radioactive, in that order (a value of 1 means the property is present, and 0 means it is not).

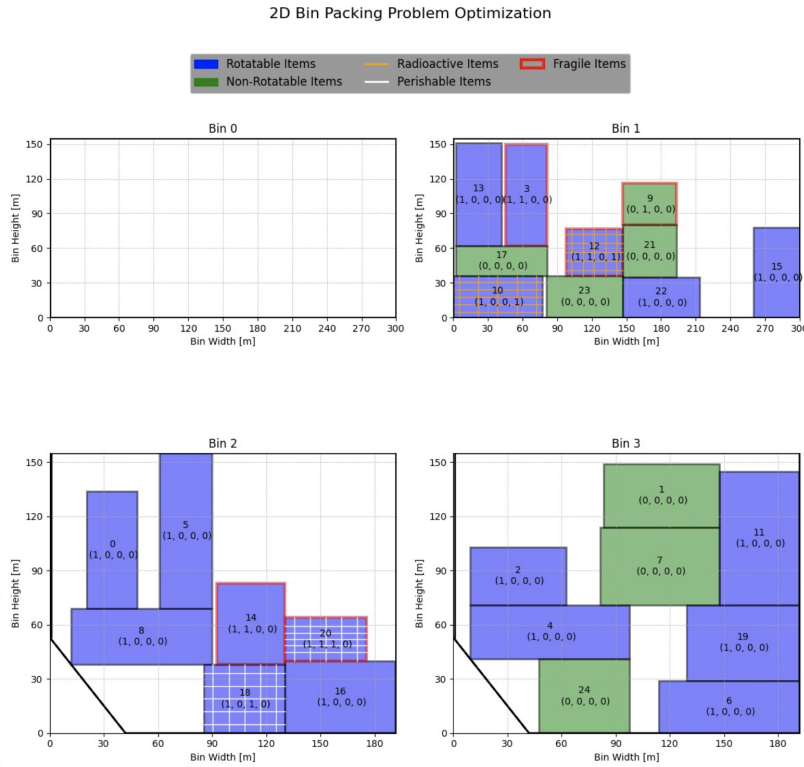


Figure 1: Best solution of the packing problem after two hours. The tuple beneath each item number indicates the following item variables (in the same order): (Rotation, Fragility, Perishability, Radioactivity).

This solution satisfies all the major **constraints** as follows:

- Each **item** is assigned to only one **bin** (C1).
- The total **area** of the **bins** assigned to each used **bin** is smaller than the **area** of the **bin** itself (C3).
- Since the original **items** are **longer** than **taller**, it can be verified that the **non-rotatable items** have remained in a **horizontal** position and have not been **rotated** (C6 and C7).
- There is no **overlap** between **items**, as all the **borders** of each **item** can be seen in the **plot** (C8, C9, C10, and C11).
- All **items** located in a **bin** with a **cut** are positioned above the **cut**.
- Each **item** is supported by **two points**, which may be the **ground**, another **item**, or the **cut** (C13).
- The **fragility** constraint is met, as no **fragile item** serves as the **support** for another **item** (C22).
- **Perishable** and **radioactive items** are not mixed in the same **bin** (C23).

These results align with expectations. The selected bins—Bin 1, Bin 2, and Bin 3—yield a total cost of 500 €. Among them, Bins 0 and 1 are the larger and more expensive options (300 € each), while Bins 2 and 3 are smaller and more economical (150 € each). The chosen configuration is logical, as it prioritizes the use of cheaper bins while still utilizing a larger one. However, Bins 2 and 3 remain relatively underutilized, prompting the question: with additional computation time, could all items be efficiently packed into just Bin 2 and Bin 3, reducing the total cost to 300 €.

The total surface area of all items is 58,380 square units, while the combined surface area of Bin 2 and Bin 3 is 59,520 square units. This suggests that, in theory, it might be possible to redistribute the items currently placed in Bin 1 into Bins 2 and 3. This could also adhere to the perishability/radioactivity constraint, as, irrespective of the amount of area, a minimum of two bins is required to separate the radioactive and perishable items. However, achieving such an optimized packing arrangement would likely require significantly more computational resources and time and given the cut and the rectangular shape of the items, it looks almost impossible. An alternative solution could involve using Bin 0 and Bin 1, which together provide a total capacity of 93,000 square units at a cost of 400 €. This configuration offers a larger margin, allowing for more flexibility in accommodating item dimensions while still ensuring all other constraints are met, e.g. fragility and rotation. Furthermore, this option would require fewer computational resources and could be obtained in a shorter amount of time compared to the more theoretically optimal solution of 300.

4 Algorithmic Results

The main algorithmic results of the optimization process are presented in [Table 1](#):

Metric	Value
Final Optimality Gap	40%
Best Feasible Solution (Objective Value)	500
Best Known Bound	300
Nodes Explored	464,217
Simplex Iterations	143,572,881
Total Computation Time	7200 s (2 hours)
Time to First Solution	216 s

Table 1: Algorithmic results of the optimization process.

The solution obtained is not guaranteed to be optimal, as the final optimality gap remains at 40%. The best feasible solution found has an objective value of 500, while the best known bound remains at 300, indicating that a better solution might exist but was not discovered within the provided time limit. It must also be considered that not many combinations of bins can be done between 500 and 300, and that obtaining a solution of 300, given the cut and the shape of the items, is very demanding. Only the combinations of two big bins with a total cost of 400 or one big and one small with a total cost of 350 seem theoretically possible. This would reduce the optimality gap to 30%, meaning that the solution obtained is closer to the optimal one.

During the optimization, the solver faced difficulties in closing the gap, likely due to the complexity of the problem, the vast search space, or the effectiveness of the applied cutting planes. In total, 464,217 nodes were explored, with 143,572,881 simplex iterations performed over 7200 seconds (2 hours). However, the first solution was obtained after just 216 seconds, which is relatively fast. The process skipped the option of combining two big bins and one small bin with a total cost of 550. This could have occurred due to the solver's focus on more promising configurations or limitations imposed by the problem's constraints, leading to the exclusion of this potentially viable option.

The solver reached a solution count of 2: one with an objective value of 500 and another with 700. However, the gap between the best solution and the best bound remains, suggesting that further improvements could be achieved by increasing computational time, refining the model, or applying more advanced heuristics. The model could also be improved by adding extensions that do not consider symmetric solutions, which can help reduce computational time.

Several parameters were adjusted throughout the project to enhance the performance of the Gurobi optimization. Specifically, the L and H values were set equal to a large number M instead of using $L = \max(L_j)$ and $H = \max(H_j)$, as this yielded improved results. Through further experimentation, this M value was set to 10,000, as it provided the best balance between solution quality and computational efficiency. In general, it is recommended to use value for M determined by the physical bounds of the domain (e.g. $L = \max(L_j)$ and $H = \max(H_j)$), as large values for M can lead to instabilities. But as this simply produced to worse results, a larger (but more arbitrary) value of 10,000 was used.

In [section 6](#), by adding an extension that improves the stability of items, the algorithmic results could be significantly different and may either involve more time or not.

5 Managerial Considerations

The 2D Bin Packing Problem is a useful mathematical abstraction for cargo allocation inside bins. However, when applied to the real world, several assumptions and simplifications can limit its applicability. Some of the main limitations are:

- **Weight-bearing capacity:** When allocating items, fragile cargo is not strategically placed, even though no other items can be stacked on top of it. Additionally, heavy cargo is neither considered nor positioned strategically.
- **Weight and balance considerations:** Items are allocated based purely on geometric fit, without regard for weight distribution. Moreover, the constraints do not incorporate the operational limits of the center of gravity.
- **Shape irregularity:** In this problem, cargo is assumed to be perfectly rectangular. However, in reality, cargo items are often non-rectangular and require specialized containers with unique shapes. Similarly, the model also pre-determines only two orthogonal rotation possibilities. With more irregular shaped items being relevant in reality, this constraint is no longer useful.
- **Cut stability:** In this assignment, the cut is treated as a support without considering whether it can actually support each item or the stability it provides. In reality, some items might require a more stable base, or their shape may not be supported by this type of cut.
- **Handling Constraints:** Although the separation constraints for perishable and radioactive items are included, they do not account for the special placement requirements or conditions of the bins.
- **Operational feasibility:** The model assumes that if an item fits, it can be loaded. However, some solutions may not be operationally feasible if cargo needs to be accessible for unloading at transit stops. Furthermore, the model assumes fully deterministic knowledge of the information about the items and doesn't account for any potential variations, e.g. size variations due to excessive packaging or incorrect measurements.

While the 2DBPP provides a useful foundation for understanding cargo allocation, its assumptions significantly oversimplify many of the complexities found in real-world scenarios. The omission of factors like the third dimension, weight-bearing capacity, and operational feasibility reduces its practicality for actual air cargo logistics. Although these simplifications make the problem more computationally manageable, they overlook critical aspects of cargo handling, such as safety, accessibility, and the unique requirements of fragile or irregularly shaped cargo. As a result, these omissions could lead to inefficiencies or operational challenges when attempting to apply the model to a real-world air cargo case study.

6 Extension

To more accurately emulate real-life scenarios, the models was extended to ensure more realistic stability of items. In the initial model, an item k can support item i if two requirements are met:

1. The upper vertical coordinate of item k is equal to the lower vertical coordinate of item i : $z_i = z'_k$
2. If the left vertex of item i is supported, the right horizontal coordinate of item k must be larger or equal than the left coordinate of item i ($x'_k \geq x_i$) OR if the right vertex of item i is supported, the left coordinate of item k must be smaller or equal to the right coordinate of item i ($x_k \leq x'_i$).

For the second requirement, overlap is not necessary. In practice this would not suffice. Additional constraint were hence introduced which ensure that there must be a minimum overlap of 20% of the width of the supported item i .

$$x'_k \geq x_i + 0.2(x'_i - x_i) - M(1 - s_{ik}) \quad \forall i, k \in J, \quad i \neq j \quad (39)$$

$$x'_i \geq x_k + 0.2(x'_i - x_i) - M(1 - s_{ik}) \quad \forall i, k \in J, \quad i \neq j \quad (40)$$

The Equation 39 targets the overlap on the left vertex of item i . It indicates that if item i supports k , then the right coordinate of i must be larger then the left coordinate of k plus 20% of the width of i . In a similar manner, Equation 40 ensures this overlap for the right vertex of item i .

In Figure 3 and Figure 2, the solution without (same as Figure 1) and with extension, respectively, can be found. First inspecting the solution without 20% overlap constraint, it can be seen that items 10 does not

satisfy the 20% requirement. Table 2 shows the overlaps for item 10 to be 5 and 8% for the left and right side, respectively. Visually, it is hard to tell if the left support of item 9 does suffice but from Table 2 it can be seen that it does as the overlap is 22%. In practice, even if it was lower the overlap was less than 20%, this would not be an issue considering that the right side is already supporting 73% of the box. This is something the extension also does not take into account, i.e. that if a significant portion (e.g. past 50%) of the box is already supported on one side, the requirement on the other side may be relaxed.

Inspecting the graph with the extension implemented clearly shows that all boxes are properly supported for at least 20% of both sides. Although visual verification should suffice, Table 2 contains the overlaps for the items that are not supported across 100% of the lower side.

Model	Item no.	Overlap L	Overlap R	Method
w/o Ext.	4	0 (<i>cut</i>)	56.8%	B&B
	7	24.2%	27.3%	B&B
	8	0 (<i>cut</i>)	5.6%	Heur.
w/ Ext.	12	20.0%	20.0%	B&B
	23	0 (<i>cut</i>)	58.0%	Heur.

Table 2: Overlap comparison with and without extension for the left side (L) and right side (R) of an item.

The extension leads to a similar suboptimal result. The same set of bins are used, thus incurring the same cost (best incumbent, best bound and gap optimality remain unchanged). This is not surprising considering the subtle nature of the extension. After all, the model without the extension already meets the 20% overlap constraint for all but one item. A notable difference between the two models is how they were obtained. As can be seen in Table 3, the model without the overlap constraint found the first feasible solution very quickly in just 216 seconds, while implementing the extension led to an increase in time of nearly 2000 seconds. Contrarily, finding an improved solution took the first model another 4500 seconds while it took the second model just 100 seconds and another 200 seconds to find the final solution cost of 500. Note that the model with extension found an intermediate solution of a cost of 550 before it found the final solution, while the initial model did not do this. This is likely related to why the solver could not find an improved solution sooner. Overall, the extended model performs more effectively as it reaches the same solution in less time but whether this is also the case when the models are run up to optimality would need to be investigated with longer run times.

The lower run time for the extended model could perhaps be explained due to the more constrained solution space. Adding the constraint removes a set of solutions that would otherwise be possible, thereby decreasing the number of solutions to be explored by the optimizer. This unfortunately does not explain why the initial model finds the first feasible solution sooner.

Solution Cost	w/o Ext.		w Ext.	
	Time [s]	Method	Time [s]	Method
700	216	B&B	2158	B&B
550	—	—	2258	Heur.
500	4771	Heur.	2435	B&B

Table 3: Time to find solutions with and without extension

2D Bin Packing Problem Optimization

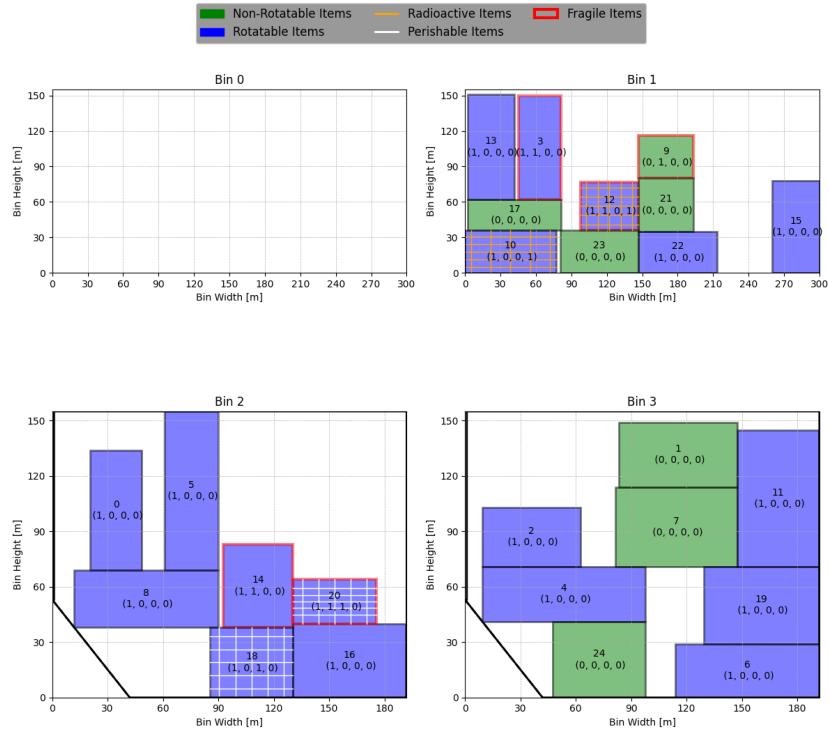


Figure 2: Solution **without** 20% overlap extension. The tuple beneath each item number indicates the following item variables (in the same order): (Rotation, Fragility, Perishability, Radioactivity)

2D Bin Packing Problem Optimization

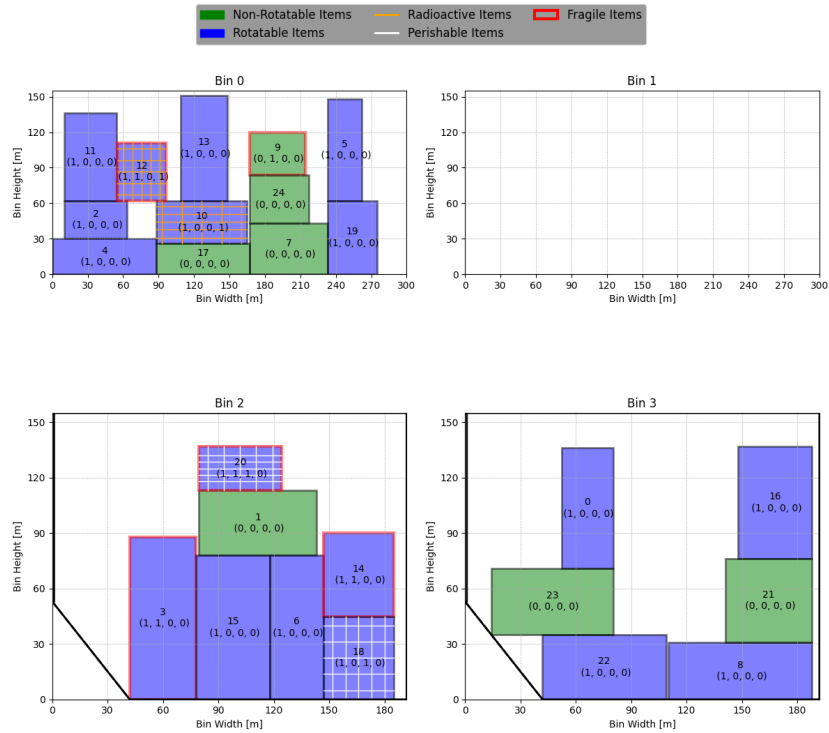


Figure 3: Solution **with** 20% overlap extension. The tuple beneath each item number indicates the following item variables (in the same order): (Rotation, Fragility, Perishability, Radioactivity)

7 Personal Reflections

7.1 Job

The worst part of the assignment was performing all the bug fixing to get the model to actually output a feasible solution within two hours as it was quite tedious. We encountered two main problems: getting the model to place items in another bin once the first one was filled and once this was fixed getting it to find a feasible solution in a normal amount of time. The first issue ended up being due to a small typo, but especially the second issue was very time-consuming as it was difficult to pinpoint what the cause was. At some point this became quite demotivating. The most enjoyable part (besides obtaining a feasible final result) was adapting the 3D model to a 2D case. I am in the Flight Performance track, so I rarely work with mathematical models in such a way. By requiring adaptations of the model you gain a much better understanding of the model itself.

- **Drop:** Overall, I really enjoyed the assignment. I have genuinely been trying to think of something I would drop but nothing comes to mind. Outside of the assignment, I would drop the homework grading weight. In my opinion, it doesn't add much besides. I presume the main goal is to incentivize following the lectures during the quarter but because the questions are relatively easy and there are only a few per quiz, it's easy to quickly look up the answer in the slides without actually studying the lecture.
- **Keep:** I think the optional addition of extensions is great as it allows you to further dive into a certain topic based on your interests or needs (e.g. if your model is very slow, focusing on algorithmic improvements. Or if your model leads to unrealistic results, focusing on extra constraints that improve this)
- **Add:** This was my first time using Gurobi, and after obtaining initial results, the optimizer felt somewhat like a black box. Identifying the causes of infeasibility and debugging errors was quite challenging. It would be helpful to perhaps include a short 20-minute introduction to Gurobi in one of the lectures, covering aspects such as constraint debugging, interpreting solver output, and handling infeasibilities. Alternatively, a short manual or a document with a list of tips would work too. Secondly, it might be interesting to prompt students to investigate the effects of the different Gurobi optimizer settings. E.g. changing the heuristics setting or the solver's algorithm.
- **Improve:** Although questions could be asked via an email there is some room for improvement for getting help with the assignments. There is quite some uncertainty on when you'll get a response on an email and it's more difficult to clearly explain issues in an email. It would perhaps be beneficial to organize one or two lunch sessions near the assignment's deadline where students can come by with questions.

7.2 Marianna

The best part of the assignment was programming the mathematical model and adapting it from the research paper. In SAT we get to program a lot of mathematical models but this specific model was very satisfying because of its visual nature. During debugging you can see what specifically is going wrong, so it is less frustrating than other models where the errors are less obvious. It is nice to see the solver come up with the different packing solutions depending on the number of items. At first I was less excited, as I wanted to try a Gate Assignment or Crew Planning problem as I found them initially more interesting, but in the end I really enjoyed the process of adapting the model from the paper and debugging using visualization. The worst part was that the amount of items was so large the solver needed a lot of time, which is tedious.

- **Drop:** I would not drop anything from this assignment, I think it is an appropriate work load and size, and very interesting. I also agree however that the homework assignments were easy to look up and because the solutions were never handed out, they could not easily be used for exam revision.
- **Keep:** I would keep the concept of adapting a model from the research paper. I think it is a creative way to start an assignment, rather than just on lecture material. I also liked how several research papers were given for inspiration on the extensions. I would also keep the pickle files, I had not used it before but it is a really nice way to store data.
- **Add:** I would add more original constraints, or perhaps different cuts or more extensions to make it more realistic, as the more creative it gets the more you learn.
- **Improve:** I would add a session on Gurobi for the non-SAT students, as I had to explain a lot to friends, and it is a relatively simple concept but an introduction would definitely help.

7.3 Júlia

The best part of the project was the process of creating a model to solve the problem as well as adapting the version from the paper. I am doing the Sustainable Air Transport profile of the master's program, so this is not the first time I have used Gurobi. Applying this tool to practical problems is always interesting. However, the worst part was the time spent running the solver multiple times without obtaining solutions and, as always happens with programming, debugging issues—such as when the optimization was using only one bin. Additionally, by limiting the runtime to two hours, we could not obtain optimal solutions, although we could infer the optimal outcome based on the optimization process.

- **Drop:** There is nothing in particular I would drop; overall, it is a useful project for understanding one of the problems discussed in the lectures.
- **Keep:** Including extra constraints, such as those for perishable or radioactive items, not only helps in understanding how to implement constraints but also provides a better understanding of the other constraints used in the paper. Additionally, the problem is highly visual, making it easier to visualize the solution, improve it, or identify errors compared to less visual problems.
- **Add:** As I have already mentioned, the extra constraints have been the most interesting part for me. Therefore, adding more constraints to make the problem even more realistic could be beneficial. However, I acknowledge that this would require additional time, and given that making the optimizer work is already quite time-consuming, it should be balanced accordingly.
- **Improve:** One aspect that could be improved is the deadline. While it is true that the assignment can be started before Christmas, in practice, if the deadline is set after the exams with a two-week window, most students end up working on it only after the exams. Additionally, completing the assignment before the exams can sometimes be beneficial for better understanding the theory required for the exam.

References

- [1] Paquay, C., Schyns, M., and Limbourg, S. (2016). A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application. *International Transactions in Operational Research*, 23(2):187–213. Received 31 May 2013; revised 9 May 2014; accepted 30 May 2014.